

gharc: A stream-and-filter tool for the GitHub Archive on consumer hardware

Arav Panwar ¹

¹ Independent Researcher

DOI: [N/A](#)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: 01 January 1970

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

gharc is a command-line tool and Python library that filters the GitHub Archive (GHArchive) dataset on consumer hardware. Rather than downloading hours of compressed events to local disk before processing, gharc streams each archive file through memory, decompresses it on the fly, applies user-specified repository and event-type filters, and writes only the matching events to Parquet or JSONL. Peak storage stays bounded by the downloads in flight at once, one temporary file per worker (each an hourly archive of roughly 60 to 150 MB in 2024), so disk use scales with the worker count rather than with the time range processed. The intended audience is software-engineering researchers, students, and small teams who do not have access to institutional warehouses or commercial cloud quotas but who need event-level GitHub data at multi-year time scales.

Statement of need

The GitHub Archive ([Grigorik, 2012](#)) is one of the most widely used data sources in mining software repositories (MSR) research, capturing nearly every public event on GitHub since February 2011. The dataset is published as hourly gzipped JSONL files. Each hour in 2024 averages roughly 90 MB compressed and several hundred megabytes uncompressed; the full archive exceeds several petabytes uncompressed.

This scale poses a practical barrier to anyone without institutional infrastructure. Three traditional approaches each exclude part of the research community:

1. **Bulk local downloads.** Storing even a single year of GHArchive uncompressed exceeds the disk budget of most laptops. Filtering after the fact still requires the full download to be present.
2. **Cloud warehouses.** GHArchive is mirrored on Google BigQuery and Snowflake ([GH Archive contributors, 2024](#)). Both are excellent for query workloads, but they require a cloud billing account.
3. **Hosted research infrastructures.** Tools such as GHTorrent ([Gousios, 2013](#)), Boa ([Dyer et al., 2013](#)), and World of Code ([Ma et al., 2019](#)) have provided shared access to GitHub-derived data, but each carries its own constraints. GHTorrent's project domain has been allowed to expire as of 2026, Boa's hosted endpoint is currently unreachable, and World of Code requires registration and remote access to dedicated servers.

gharc fills the laptop-friendly local-first niche. By streaming each hour's compressed archive through a small in-memory buffer and discarding it immediately after filtering, the tool keeps peak storage bounded while preserving the ability to operate on multi-year time ranges with no warehousing dependency. This is the configuration most useful to independent researchers, students, and small teams.

Architecture

gharc is a stream-and-filter pipeline. The user supplies a date range, an optional list of repositories, and an optional list of event types; gharc constructs the GHArchive URL for each hour in the range and dispatches them to a worker thread pool.

Each worker downloads the hour's gzipped JSONL file to a temporary location with HTTP range-resume support, then iterates over the file line by line. Lines are first filtered by a fast byte-level token check before any JSON parsing, so gharc skips the majority of irrelevant events without paying parser cost. Matching events are JSON-parsed, validated against the full filter, and yielded to the main thread.

The main thread receives matching events from completed futures and writes them to disk through a DataWriter that wraps a long-lived `pyarrow.parquet.ParquetWriter` (Apache Arrow contributors, 2024) for true streaming append. Nested fields (actor, repo, payload, org) are JSON-stringified before write so the Parquet schema is stable across heterogeneous event types within the same output file. Each worker maintains its own requests session (Reitz, 2011) in thread-local storage so HTTP connection pooling persists across the hours that worker processes.



Figure 1: Stream-and-filter architecture.

Performance

We measured gharc on a Windows 11 laptop (12 logical cores, 15 GB RAM) over a typical residential connection.

A six-hour window of GHArchive (2024-01-01 00:00 to 06:00 UTC), filtered to apache/spark, completed in 76 seconds with a single worker and 58 seconds with four workers. Both runs recovered the same 14 events, so concurrency does not affect output correctness. Peak resident set size stayed under 110 MB in both configurations. The limiting factor on residential links is HTTPS download throughput rather than CPU; additional workers beyond a small number add little once the connection saturates.

Over the same six-hour window gharc streams approximately 416 MB of compressed source data from GHArchive (six hourly files of roughly 60 to 85 MB each) but never retains it. The full source is still transferred, so this is not a reduction in bandwidth; what stays bounded is local disk. Peak local disk is held to the temporary files in flight, one per worker: about 85 MB with a single worker and about 250 MB at the default four workers. The filtered Parquet output for apache/spark over that window is 53 KB, and local disk does not grow with the length of the window processed.

Extrapolating from these numbers, a full January to June 2024 fetch (4,380 hours of source data at about 90 MB per hour) is approximately 395 GB streamed, which is achievable in three to four evening sessions on the same hardware. These figures assume a tight filter (one repository) where the byte-level token check rejects the majority of lines before JSON parsing; wider or empty filters would shift the bottleneck toward parsing cost and produce different absolute throughput. The reproducible benchmark scripts are included in the repository under benchmarks/.

Motivating use case

The motivation for gharc came from a six-month analysis of Apache Spark contributor activity originally conducted as an undergraduate course mini-project ([Panwar, 2025](#)). That earlier pipeline downloaded GHArchive month by month, ran a separate filter script over each month, and combined an off-by-one error in the date range (a “seventh-month bleed” caused by an inclusive end bound) with intermediate local disk pressure of approximately 100 GB during processing. gharc reproduces the same kind of analysis on the same laptop with no intermediate disk pressure and a single command, and treats the end bound as exclusive so the bleed cannot recur.

Related work

Several tools have addressed the GHArchive analysis problem, each with different trade-offs.

GHTorrent ([Gousios, 2013](#)) historically served as the default GitHub-mining database for MSR research and offered both periodic data dumps and a SQL-queryable mirror. As of 2026 the project’s primary domain redirects to an unrelated commercial site, illustrating the maintenance fragility of long-running shared services.

PyDriller ([Spadini et al., 2018](#)) mines git repositories directly via the local git history of cloned projects. It operates at a different layer to gharc: PyDriller exposes commits, file diffs, and developer information from a checked-out repository, whereas gharc operates on the GitHub event stream (issues, pull requests, reviews, watch events, and so on). The two are complementary; an MSR study often needs both.

World of Code ([Ma et al., 2019](#)) provides shared access to a curated cross-reference of millions of git repositories. Researchers obtain accounts and submit jobs to dedicated servers. The infrastructure is excellent for very large cross-project queries but introduces an institutional dependency.

Boa ([Dyer et al., 2013](#)) offers a domain-specific language for ultra-large-scale repository queries against curated datasets, again served from a hosted endpoint. At the time of writing the public endpoint at `boa.cs.iastate.edu` is unreachable.

Cloud-warehouse mirrors of GHArchive on Google BigQuery and Snowflake ([GH Archive contributors, 2024](#)) are highly performant but require a cloud billing account.

gharc occupies the local-first, laptop-friendly niche: no shared infrastructure, no billing account, no schema-bound DSL, just the original GHArchive files streamed and filtered on demand.

Limitations

gharc has scope boundaries researchers should plan around:

- It is bounded by HTTPS download throughput on the GHArchive side, not by local CPU. On residential connections, additional workers beyond a small number give diminishing returns.
- It is a streaming filter, not a query system. Aggregations, joins, and ad-hoc cross-time queries are deliberately out of scope; users pair gharc with whichever analysis tool they prefer (pandas, Polars, DuckDB, Spark).
- GHArchive uses the GitHub Events API schema from January 2015 onward and an older Timeline API schema before that. gharc does not normalise across that boundary; studies covering 2011 to 2014 should expect some fields to be missing or differently shaped.

- Crash-safe resume requires JSONL output. ParquetWriter cannot append to a closed file, so multi-hour runs that may need to recover should write JSONL and convert with `gharc convert` after the run.

Software availability

The source code is hosted at github.com/aravpanwar/gharc and the v0.1.0 release is archived on Zenodo (DOI: [10.5281/zenodo.19814233](https://doi.org/10.5281/zenodo.19814233)). The concept DOI [10.5281/zenodo.19814232](https://doi.org/10.5281/zenodo.19814232) always resolves to the latest archived version.

Acknowledgements

The author thanks Ilya Grigorik and the GHArchive maintainers for the public dataset on which this tool depends, and the maintainers of the requests ([Reitz, 2011](#)), pandas ([McKinney, 2010](#)), pyarrow ([Apache Arrow contributors, 2024](#)), tqdm, and orjson libraries that gharc builds on.

References

- Apache Arrow contributors. (2024). *Apache arrow*. <https://arrow.apache.org/>
- Dyer, R., Nguyen, H. A., Rajan, H., & Nguyen, T. N. (2013). Boa: A language and infrastructure for analyzing ultra-large-scale software repositories. *Proceedings of the 2013 International Conference on Software Engineering*, 422–431.
- GH Archive contributors. (2024). *GH Archive on Google BigQuery*. <https://www.gharchive.org/#bigquery>.
- Gousios, G. (2013). The GHTorrent dataset and tool suite. *Proceedings of the 10th Working Conference on Mining Software Repositories*, 233–236.
- Grigorik, I. (2012). *GH Archive: A GitHub timeline archive*. <https://www.gharchive.org/>.
- Ma, Y., Bogart, C., Amreen, S., Zaretzki, R., & Mockus, A. (2019). World of code: An infrastructure for mining the universe of open source VCS data. *2019 IEEE/ACM 16th International Conference on Mining Software Repositories*, 143–154.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51–56.
- Panwar, A. (2025). *Apache Spark codebase evolution: A six-month GHArchive analysis*. https://github.com/aravpanwar/Spark_Codebase_Evolution.
- Reitz, K. (2011). *Requests: HTTP for humans*. <https://requests.readthedocs.io>
- Spadini, D., Aniche, M., & Bacchelli, A. (2018). PyDriller: Python framework for mining software repositories. *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 908–911.